

4. O webhook faz uma requisição para a API do Asterisk (a ARI) pedindo para originar uma chamada.
5. O Asterisk liga para o ramal 1000 e, quando alguém atende, toca o áudio de alerta que já está gravado.
6. O Linphone, que é o softphone registrado nesse ramal, toca no fone. Você atende e ouve o aviso.

As peças (containers Docker)

Container	Imagem	Função
ssa-db	mariadb	Banco de dados do Zabbix
ssa-zabbix-server	zabbix-server	O cérebro: roda os checks e dispara as ações
ssa-zabbix-web	zabbix-web-nginx	Interface web de configuração (porta 8081)
ssa-asterisk	andrius/asterisk	Central telefônica que faz a ligação
ssa-website	nginx	Site de exemplo, para você testar derrubando ele

O Linphone roda no desktop, fora do Docker. Ele é o telefone que recebe a chamada.

Os conceitos do Zabbix

O Zabbix trabalha em três níveis, um puxando o outro:

- **Item:** o que medir. Aqui é o `web.test.fail`, que vale 0 quando o site está OK e 1 quando falhou.
- **Trigger:** quando isso vira problema. Aqui é `last(web.test.fail) <> 0`.
- **Action:** o que fazer quando vira problema. Aqui é chamar o webhook que liga para o telefone.

Como o Zabbix conversa com o Asterisk

A ARI (Asterisk REST Interface) é uma API HTTP do Asterisk, na porta 8088. O webhook do Zabbix faz um POST pedindo a ligação, mais ou menos assim:

```
{
  "endpoint": "PJSIP/1000",    // liga para este ramal
  "extension": "alert",       // executa esta extensão...
  "context": "site-alert",    // ...neste contexto do dialplan
  "priority": 1
}
```

O Asterisk recebe esse pedido e segue o dialplan (no `extensions.conf`): atende, toca o áudio com o Playback e desliga.

3. Pré-requisitos

- Docker e Docker Compose (v2 ou mais novo) instalados
 - Pelo menos 2 GB de RAM livres
 - Um softphone (o Linphone) instalado no desktop
 - Linux (foi testado no Kali)
-

4. Como subir o serviço

Passo 1, entrar na pasta do projeto

```
cd ~/phone-site-alert
```

Passo 2, subir tudo (jeito recomendado)

Use o script que sobe a stack e ainda prepara o softphone numa tacada só:

```
./start.sh
```

Esse script faz três coisas:

1. Sobe todos os containers (`docker compose up -d`)
2. Espera o Asterisk ficar saudável
3. Reinicia o Linphone limpo, forçando um registro novo do ramal 1000

O motivo de reiniciar o Linphone junto: o Asterisk roda em container, atrás do NAT do Docker. Quando os containers reiniciam, o registro do telefone "morre" e o Linphone, que usa UDP, só ia se registrar de novo umas horas depois. Reabrir ele força o registro na hora.

Passo 2 (jeito manual)

Se preferir subir na mão:

```
docker compose up -d      # sobe os containers
docker compose ps          # confere se estão "Up" e "healthy"
```

Depois é só abrir o Linphone.

Passo 3, acessos

Serviço	Endereço	Credenciais
Zabbix (web)	http://localhost:8081	Admin / zabbix
Asterisk (ARI)	http://localhost:8088/ari	zabbix / ChangeMe_ARI_123
Site de exemplo	http://localhost:8082	(sem login)
Ramal SIP	127.0.0.1:5062 (UDP)	1000 / ChangeMe_1000

Passo 4, configurar o monitoramento (uma vez só)

A configuração tem cinco partes, nesta ordem: web scenario, trigger, media type, mídia do usuário e action. Dá para criar tudo automático (rápido) ou na mão pelo painel (bom para entender cada peça). Faça só um dos dois.

Jeito rápido (script via API)

```
python3 scripts/zbx_setup.py
```

Cria tudo de uma vez, e pode rodar de novo sem duplicar nada: o web scenario "Check site", a trigger "Site DOWN", o media type Webhook "Asterisk Call" (já habilitado), a mídia do Admin (apontando para o ramal 1000) e a Action.

Jeito manual (pelo painel, passo a passo)

Abra o **http://localhost:8081** e entre com **Admin / zabbix**. Faça as cinco etapas na ordem.

4.1, web scenario (o que monitora o site)

1. Vá em **Data collection > Hosts**
2. Na linha do host "Zabbix server", clique na coluna **Web** (ou em *Web scenarios*)
3. Clique em **Create web scenario** (canto superior direito)
4. Na aba **Scenario**, preencha: - Name: **Check site** - Update interval: **30s** - Attempts: **1**
5. Na aba **Steps**, clique em **Add** e preencha: - Name: **home** - URL: **http://website** (ou a URL do site real que você quer monitorar) - Required status codes: **200** - Clique em **Add** para salvar o passo
6. Clique em **Add** para salvar o cenário

Isso já cria sozinho os itens de coleta, e entre eles está o **web.test.fail[Check site]**, que vale 0 quando o site responde e 1 quando ele falha.

4.2, trigger (quando considerar que caiu)

1. Ainda em **Data collection > Hosts**, na linha do "Zabbix server", clique em **Triggers**
2. Clique em **Create trigger**
3. Preencha: - **Name**: `Site DOWN: {HOST.NAME}` - **Severity**: **High** - **Expression**: clique em **Add** para montar pela tela, ou cole direto isto:

```
last(/Zabbix server/web.test.fail[Check site])<>0
```

Em português: se a última medição de falha for diferente de zero, é problema. 4. Clique em **Add**

Se o site oscilar muito e você não quiser ligação a cada piscada, troque a expressão para exigir três falhas seguidas:

```
min(/Zabbix server/web.test.fail[Check site],#3)<>0
```

4.3, media type Webhook (como a ligação é feita)

1. Vá em **Alerts > Media types**
2. Clique em **Create media type**
3. Na aba **Media type**: - **Name**: `Asterisk Call` - **Type**: **Webhook** - **Script**: cole o conteúdo do arquivo `scripts/zabbix_webhook.js` - Em **Parameters**, adicione um por um (botão Add):

Name	Value
<code>ramal</code>	<code>{ALERT.SENDTO}</code>
<code>event_value</code>	<code>{EVENT.VALUE}</code>
<code>ari_url</code>	<code>http://asterisk:8088/ari/channels</code>
<code>ari_user</code>	<code>zabbix</code>
<code>ari_pass</code>	<code>ChangeMe_ARI_123</code>

1. Deixe **Enabled** marcado. Esse é o detalhe que mais escapa: se ficar desmarcado, a ligação simplesmente não sai.
2. Clique em **Add**

4.4, mídia do usuário (para quem ligar)

1. Vá em **Users > Users** e clique no usuário **Admin**
2. Na aba **Media**, clique em **Add**: - **Type**: `Asterisk Call` - **Send to**: `1000` (o ramal que vai tocar) - **When active**: `1-7,00:00-24:00` - **Use if severity**: deixe todas marcadas
3. Clique em **Add** e depois em **Update**

4.5, action (o que junta a trigger com a ligação)

1. Vá em **Alerts > Actions > Trigger actions**
2. Clique em **Create action**
3. Na aba **Action**: - **Name**: `Ligar quando site cair` - Em **Conditions**, clique em **Add**, escolha **Type** = `Trigger` e selecione `Site DOWN: Zabbix server`
4. Na aba **Operations**: - Em **Operations**, clique em **Add**:
 - **Send to users**: `Admin`
 - **Send only to**: `Asterisk Call`
 - **Default operation step duration**: `1h` (assim ele não fica religando em loop enquanto o site está fora)
5. Clique em **Add**

Feito isso, toda vez que o site cair o telefone toca sozinho.

5. Configurar o softphone (Linphone)

Na primeira vez você precisa registrar a conta SIP no Linphone:

1. Na tela inicial, escolha **Third-party SIP account** (não é "Create account")
2. Clique em **I understand**
3. Preencha:

Campo	Valor
Username	<code>1000</code>
Password	<code>ChangeMe_1000</code>
Domain	<code>127.0.0.1:5062</code>
Transport	UDP

1. Clique em **Connection**. O status fica verde quando registra.

Cuidado com o Transport: tem que ser UDP. O Linphone vem com TLS por padrão, e com TLS não funciona neste Asterisk.

6. Como testar

Com tudo no ar e o ramal 1000 registrado:

Teste rápido (sem o Zabbix)

```
./scripts/test_call.sh          # liga e toca "site fora do ar"
./scripts/test_call.sh recovery # liga e toca "serviço restabelecido"
```

O Linphone toca, você atende e ouve a voz.

Teste completo (a cadeia automática)

```
docker compose stop website      # derruba o site, liga em uns 30s
# ... o telefone toca sozinho ...
docker compose start website     # restaura o site
```

Aqui você vê tudo acontecendo sem tocar em nada: o Zabbix percebe a queda, a Action dispara, o Asterisk liga e o Linphone toca.

7. Comandos do dia a dia

```
cd ~/phone-site-alert

./start.sh          # sobe tudo (containers + Linphone)
docker compose ps   # ver status
docker compose logs -f asterisk # logs do Asterisk em tempo real
docker compose stop website # simular queda
docker compose start website # restaurar
docker compose down  # desligar tudo (a config do Zabbix fica salva)
```

8. Resolução de problemas

Problema	Causa provável	Solução
Liga, mas não chega no Linphone	Registro SIP morreu depois de um restart (NAT do Docker)	Rode <code>./start.sh</code> , que reinicia o Linphone e registra de novo
Linphone não registra	Transport errado (TLS) ou porta errada	Use Domain <code>127.0.0.1:5062</code> e Transport UDP
Webhook falha com "Media type disabled"	Media type ficou desabilitado	Já vem corrigido no <code>zbx_setup.py</code> (criado habilitado)
Sem áudio no Linphone	Dispositivo de saída errado	Ajuste o dispositivo de áudio nas configurações do Linphone

Para ver se o ramal está registrado e respondendo:

```
docker exec ssa-asterisk asterisk -rx "pjsip show contacts"
```

O status tem que ser **Avail**. Se aparecer **Unavail** ou não aparecer nada, rode `./start.sh`.

9. Detalhe técnico: o NAT do Docker

A parte mais sensível do projeto é a conversa entre o Linphone, que está no desktop, e o Asterisk, que está num container. Esse tráfego passa pelo NAT do Docker, e isso exige dois cuidados:

O primeiro é o `qualify_frequency=30` no `pjsip.conf` do Asterisk. Com ele, o Asterisk manda um ping (um OPTIONS) para o ramal a cada 30 segundos. Isso mantém o caminho do NAT vivo, porque senão ele expira por inatividade, e de quebra o Asterisk percebe na hora quando o ramal sai do ar.

O segundo é o `start.sh`, que reinicia o Linphone junto com a stack. Assim o registro do ramal sempre nasce novo e qualificado quando tudo sobe.

Esses dois detalhes juntos resolvem aquele sintoma chato de "ligou mas a chamada não chega".

10. Compatibilidade (Ubuntu, Debian e outras distros)

A maior parte do projeto roda em Docker, então é quase agnóstico de distro. Toda a stack (Zabbix, Asterisk, nginx e mariadb) sobe em container e funciona igual em Ubuntu, Debian, Fedora, Arch ou qualquer Linux que tenha Docker Engine e Docker Compose v2. O projeto foi montado no Kali, que é Debian por baixo, então Ubuntu e Debian são os casos mais parecidos e rodam sem ajuste nenhum na stack.

O que muda de uma distro para outra são só as partes que rodam no host, fora do Docker.

O softphone (Linphone). É um AppImage e roda em qualquer distro, mas precisa do FUSE2. No Debian e Ubuntu mais antigos o pacote é `libfuse2`; no Ubuntu 24.04 em diante ele virou `libfuse2t64`.

```
sudo apt install libfuse2      # ou libfuse2t64 no Ubuntu 24.04+
```

A geração dos áudios. Só importa se você quiser regerar as vozes, porque os áudios já vêm prontos no repositório. Se quiser mexer, no Ubuntu e Debian:

```
sudo apt install espeak-ng mbrola mbrola-br4 sox libsox-fmt-all
```

No Ubuntu, o `mbrola-br4` está no repositório `universe`, então pode ser preciso habilitar ele antes.

O `start.sh` . Usa `bash` , `kill` , `nohup` , `ss` e `docker` , que são padrão em Ubuntu e Debian. Roda sem ajuste.

Usando como servidor, sem desktop

Se a ideia for usar de verdade e não como laboratório no desktop, o telefone que recebe a ligação nem precisa estar na mesma máquina. Pode ser um app no celular (Linphone ou Zoiper), outra máquina na rede ou um ramal físico SIP. Nesse cenário some toda a parte de FUSE, áudio e interface gráfica, e sobra só a stack Docker, que roda num Ubuntu Server ou Debian sem tela tranquilamente.

Phone Site Alert, laboratório de monitoramento com alerta por voz.